

```

1  r"""prelude-doctests.py -- every prelude name explained and demonstrated.
2
3  The one file in the course allowed an import, because the prelude IS its subject. One entry per name, in
4  prelude.py's own order. Each entry is a short explanation -- what it does, what the arguments mean, when
5  to reach for it, and the gotcha if there is one -- followed by several doctest examples, so the whole
6  reference is executable:
7
8      python3 prelude-doctests.py          # runs every example
9
10 Two conventions to load before reading. First, argument order is Haskell's: the FUNCTION comes first, the
11 data second -- map_(f, xs), sortOn(key, xs). Second, inside a fold the combiner's own arguments differ by
12 direction: foldl's f takes (accumulator, element); foldr's f takes (element, accumulator). When a fold
13 misbehaves, check that order first.
14
15 ===== 1. combinators =====
16
17 id -- the identity function: returns its argument unchanged. Sounds useless; is everywhere. It is the "do
18 nothing" you pass where a transformation is required -- a sort key meaning "the value itself", the seed
19 of compose.
20 >>> id(42)
21 42
22 >>> sortOn(id, [3, 1, 2])
23 [1, 2, 3]
24
25 const -- freezes a value into a function: const(x) returns a function that ignores its argument and
26 always answers x. Use it to fill a callback slot with a constant.
27 >>> five = const(5)
28 >>> five("anything"), five(99)
29 (5, 5)
30 >>> map_(const('*'), [1, 2, 3])
31 ['*', '*', '*']
32
33 flip -- swaps the two arguments of a function. An adapter: the function you have takes (a, b), the slot
34 you are filling supplies (b, a). This is how the prelude derives foldr from foldl.
35 >>> sub = lambda a, b: a - b
36 >>> sub(10, 1), flip(sub)(10, 1)
37 (9, -9)
38 >>> flip(pow)(3, 2)          # pow(2, 3)
39 8
40
41 How it works: flip(f) returns a new function that hands its two arguments to f in the opposite order.
42 Nothing is computed at flip time -- it is pure plumbing, applied when the data arrives in the "wrong"
43 order for the function you already have.
44 >>> pair = lambda a, b: (a, b)
45 >>> flip(pair)(1, 2)
46 (2, 1)
47
48 Why it earns its place: the prelude's own foldr is the showcase. foldl's combiner takes (accumulator,
49 element); a right fold's combiner takes (element, accumulator). Rather than build a second fold machine,
50 foldr = foldl of the FLIPPED combiner over the reversed list -- one adapter turned an existing engine
51 into its mirror image.
52 >>> f = lambda x, acc: '(' + x + acc + ')'
53 >>> foldr(f, "abc", "") == foldl(flip(f), list(reversed("abc")), "")
54 True
55
56 Remember it as: when argument order is the only thing wrong, adapt -- don't rewrite.
57
58 curry -- splits a two-argument function into two one-argument stages, so you can supply the first
59 argument now and the second later.
60 >>> add = lambda a, b: a + b
61 >>> curry(add)(1)(2)
62 3
63 >>> add_ten = curry(add)(10)
64 >>> add_ten(5)
65 15
66
67 How it works: read the three lambdas inside-out, then watch a call peel one layer at a time. Each call
68 binds ONE argument by closure -- after curry(add)(10), the 10 lives in the returned function's captured
69 environment, and that half-applied function is a value you can name, store, and pass.
70 >>> curry(add)(10)(5)
71 15
72 >>> add_ten = curry(add)(10)
73 >>> add_ten(5), add_ten(90)
74 (15, 100)
75
76 Why it earns its place: the prelude's pipelines -- map_, filter_, find, compose -- all want ONE-argument
77 functions in their function slot, while real logic usually takes two ("multiply by k", "is x greater than
78 n?"). Currying is the adapter: close over the configuration argument now, hand the pipeline the unary
79 stage it wants.
80 >>> gt = curry(lambda n, x: x > n)
81 >>> filter_(gt(10), [4, 11, 8, 40])
82 [11, 40]
83 >>> scale = curry(lambda k, x: k * x)
84 >>> map_(scale(3), [1, 2, 3])
85 [3, 6, 9]
86
87 The Haskell connection: in Haskell EVERY function is curried automatically -- add :: Int -> Int -> Int
88 really means Int -> (Int -> Int), so add 10 is ordinary code and map (add 10) xs falls out for free.

```

```

89 Every arrow in a Haskell type is another layer of this doll.
90
91 Versus partial: both bind early. partial binds any number of arguments, all at once, when you have them;
92 curry restructures the function so binding happens one call at a time, decided later by whoever holds it.
93 In Python, partial is the cheaper everyday tool; curry is the pipeline-shaper and the reading key for
94 Haskell signatures.
95
96 Remember it as: curry turns one two-argument function into two one-argument functions -- and one-argument
97 functions are the only thing pipelines eat.
98
99 Roll your own: three lambdas, each returning the next -- every layer is a closure capturing one more
100 argument until the innermost body finally has both. No state, no storage: the captured environment IS the
101 data structure.
102
103 uncurry -- the reverse adaptation: a two-argument function becomes a one-argument function of a PAIR.
104 Reach for it when your data is already (a, b) tuples and your function is not.
105 >>> uncurry(add)((1, 2))
106 3
107 >>> map_(uncurry(lambda a, b: a * b), [(2, 3), (4, 5)])
108 [6, 20]
109
110 partial -- freezes any number of leading arguments, returning a function that takes the rest. Like curry
111 but takes the arguments now, all at once.
112 >>> partial(pow, 2)(10) # pow(2, 10)
113 1024
114 >>> clamp_floor = partial(max, 0) # "never below zero"
115 >>> clamp_floor(-5), clamp_floor(7)
116 (0, 7)
117
118 on -- builds a two-argument function that first sends BOTH arguments through a key function: on(f,
119 key)(x, y) == f(key(x), key(y)). Its natural habitat is comparators: compare two records by one field.
120 >>> on(add, len)("ab", "c") # len("ab") + len("c")
121 3
122 >>> same_length = on(lambda a, b: a == b, len)
123 >>> same_length("abc", "xyz"), same_length("ab", "abc")
124 (True, False)
125 >>> sortBy(on(lambda a, b: a - b, snd), [(1, 9), (2, 3)])
126 [(2, 3), (1, 9)]
127
128 How it works: on(f, key) builds a two-argument function that routes BOTH inputs through key before
129 combining them with f: on(f, key)(x, y) == f(key(x), key(y)). Combination logic and projection logic stay
130 separate, so each is reusable on its own.
131 >>> shorter = on(lambda a, b: a < b, len)
132 >>> shorter("ab", "abc"), shorter("abc", "ab")
133 (True, False)
134
135 Why it earns its place: comparator factories. Any "compare records by one field" collapses to on(compare,
136 field) -- Haskell's sortBy (compare 'on' snd) is exactly sortBy(on(cmp, snd)) here, and reads as its own
137 documentation.
138 >>> sortBy(on(lambda a, b: a - b, snd), [('a', 9), ('b', 1), ('c', 5)])
139 [('b', 1), ('c', 5), ('a', 9)]
140
141 Versus sortOn: when a per-element key exists, sortOn(key, xs) is simpler and faster (the key runs once
142 per element). on shines when an API demands a genuine two-argument comparator, or when the combining
143 function is something other than comparison.
144
145 Remember it as: compare the keys, not the things.
146
147 NOTHING -- a private sentinel object meaning "no argument was supplied". Why not None? Because None is a
148 legitimate VALUE a caller might pass; NOTHING can only mean "omitted", since nobody else has a reference
149 to it. Always test with 'is'. This is how foldl and accumulate tell an absent initial value from a real
150 one.
151 >>> x = NOTHING
152 >>> x is NOTHING
153 True
154 >>> def greet(name=NOTHING):
155 ...     return "hello, stranger" if name is NOTHING else "hello, " + name
156 >>> greet()
157 'hello, stranger'
158 >>> greet("ada")
159 'hello, ada'
160
161 How it works: object() creates a fresh, featureless object whose only property is its IDENTITY -- nothing
162 else in the program can ever be it. Testing x is NOTHING therefore asks exactly one question: "was I
163 handed this precise sentinel?", which makes it a safe flag for "no argument was supplied".
164
165 Why not None, demonstrated: None is data (fromMaybe's whole job is handling it), so using it to also mean
166 "argument omitted" would blur two different situations. The sentinel keeps them apart: None is Nothing in
167 the DATA; NOTHING is Nothing in the ARGUMENT LIST.
168 >>> def describe(x=NOTHING):
169 ...     if x is NOTHING:
170 ...         return "no argument"
171 ...     return "got " + repr(x)
172 >>> describe()
173 'no argument'
174 >>> describe(None)
175 'got None'
176

```

```

177 Where the prelude leans on it: foldl and accumulate take an optional seed. With NOTHING as the default
178 they can tell "seed omitted -- use the first element" from "the seed IS None", which a None default could
179 not.
180
181 Remember it as: None is a value; NOTHING is the absence of one.
182
183 fromMaybe -- lands a default: fromMaybe(d, x) is x unless x is None, in which case it is d. The standard
184 way to finish a pipeline built on the None-is-Nothing convention (find, lookup, getpath, stripPrefix all
185 return None on failure).
186 >>> fromMaybe(0, None), fromMaybe(0, 5)
187 (0, 5)
188 >>> fromMaybe('_', find(lambda c: c == 'q', "abc"))
189 '_'
190 >>> fromMaybe(99, lookup('b', [( 'a', 1), ('b', 2)]))
191 2
192
193 Start from a problem you already have: find looks for something that might not be there, and when it is
194 not, you are left holding None -- which you cannot print to a user, add, or put in a report. So you write
195 a backup plan:
196 >>> result = find(even, [3, 5, 7])
197 >>> if result is None:
198 ...     result = 0
199 >>> result
200 0
201
202 fromMaybe is those three lines as one word. Argument order: the backup plan first, then the risky thing
203 -- said aloud it parses: "use 0 if this comes back empty".
204 >>> fromMaybe(0, find(even, [3, 5, 7]))
205 0
206 >>> fromMaybe(0, find(even, [3, 5, 8]))
207 8
208
209 Versus x or d: Python's or falls back on ANYTHING falsy -- 0, "", [], False -- not just None, so it
210 silently destroys legitimate zero-ish answers. fromMaybe asks the only correct question: is this the
211 absence marker?
212 >>> fromMaybe(99, 0), (0 or 99)
213 (0, 99)
214
215 Remember it as: ifNoneUse -- the three-line backup plan as one word; or-with-a-conscience.
216
217 isJust -- the None test as a predicate you can hand to other tools. The lambda it replaces,
218 'lambda v: v is not None', kept reappearing wherever a Maybe-producing step feeds a predicate slot;
219 naming it also locks in the discipline: identity against None, never truthiness, so 0 and '' count as
220 present.
221 >>> isJust(0), isJust(''), isJust(None)
222 (True, True, False)
223 >>> find(isJust, [None, None, 7, None])
224 7
225 >>> all(map(isJust, [1, 0, 'x']))
226 True
227
228 isNothing -- the mirror: True only for None. Counting failures reads as one word.
229 >>> isNothing(None), isNothing(0)
230 (True, False)
231 >>> len(filter_(isNothing, [None, 3, None]))
232 2
233
234 Together they finish the Maybe family: fromMaybe LANDS an optional value, mapMaybe/catMaybes FILTER
235 optional values, isJust/isNothing TEST one. Haskell's Data.Maybe has all five under the same names.
236 >>> readings = [(1, None), (2, 7.5), (3, None)]
237 >>> fromMaybe(-1, find(isJust, map(snd, readings)))
238 7.5
239
240 ===== 2. pairs =====
241
242 fst -- the first element of a pair. Reads better than p[0] in code that is all tuples, and slots into
243 higher-order positions.
244 >>> fst((1, 2))
245 1
246 >>> map_(fst, [(1, 'a'), (2, 'b')])
247 [1, 2]
248
249 snd -- the second element of a pair. The classic sort key for (key, count) items.
250 >>> snd((1, 2))
251 2
252 >>> sortOn(snd, [( 'a', 3), ('b', 1)])
253 [( 'b', 1), ('a', 3)]
254
255 swap -- exchanges a pair's elements. One honest use: inverting a dict by swapping its items.
256 >>> swap((1, 2))
257 (2, 1)
258 >>> dict(map_(swap, [( 'a', 1), ('b', 2)]))
259 {1: 'a', 2: 'b'}
260
261 ===== 3. arithmetic & logic =====
262
263 succ -- successor: one more; on a character (Haskell's Char instance of Enum), the next codepoint.
264 >>> succ(41)

```

```

265 42
266 >>> succ('a')
267 'b'
268 >>> map_(succ, [1, 2, 3])
269 [2, 3, 4]
270
271 pred -- predecessor: one less; characters step back one codepoint.
272 >>> pred(43)
273 42
274 >>> pred('b')
275 'a'
276
277 even -- is n divisible by two? A predicate shaped for filter_ and friends.
278 >>> even(4), even(7)
279 (True, False)
280 >>> filter_(even, [1, 2, 3, 4])
281 [2, 4]
282
283 odd -- the complement of even.
284 >>> odd(4), odd(7)
285 (False, True)
286 >>> partition(odd, [1, 2, 3, 4])
287 ([1, 3], [2, 4])
288
289 not_ -- logical negation as a function ('not' is a keyword, so it cannot be passed around; not_ can).
290 Composes predicates.
291 >>> not_(True)
292 False
293 >>> is_odd = compose(not_, even)
294 >>> is_odd(3)
295 True
296
297 otherwise -- just True, wearing Haskell's name for the catch-all guard. Its habitat is the RULES
298 TABLE: a long if/elif ladder rewritten as data -- (predicate, result) pairs tried in order, find
299 picking the first match. One care: Haskell guards are EVALUATED, so bare otherwise works there; a
300 table's guards are APPLIED to the argument, so wrap it as const(otherwise) -- the guard that ignores
301 its input and says yes. Result slots follow the same rule: fixed answers wear const, computed ones
302 (like str below) go bare.
303 >>> otherwise
304 True
305 >>> RULES = [(lambda n: n % 15 == 0, const("FizzBuzz")),
306 ...         (lambda n: n % 3 == 0, const("Fizz")),
307 ...         (lambda n: n % 5 == 0, const("Buzz")),
308 ...         (const(otherwise), str)]
309 >>> speak = lambda n: snd(find(lambda rule: fst(rule)(n), RULES))(n)
310 >>> map_(speak, [1, 3, 5, 15])
311 ['1', 'Fizz', 'Buzz', 'FizzBuzz']
312
313 signum -- the sign of a number as -1, 0 or 1, computed by arithmetic on two comparisons -- no branches.
314 Also a ready-made comparator result.
315 >>> signum(-7), signum(0), signum(3)
316 (-1, 0, 1)
317 >>> sortBy(lambda a, b: signum(a - b), [3, 1, 2])
318 [1, 2, 3]
319
320 div -- integer division that FLOORS (rounds toward negative infinity), like Haskell's div and Python's
321 //. The div/mod pair and the quot/rem pair agree on positives and split on negatives -- interviews live
322 in that gap.
323 >>> div(7, 2), div(-7, 2)
324 (3, -4)
325
326 mod -- the remainder that matches div: its sign follows the DIVISOR, and div(a,b)*b + mod(a,b) == a
327 always holds.
328 >>> mod(7, 2), mod(-7, 2), mod(7, -2)
329 (1, 1, -1)
330
331 quot -- integer division that TRUNCATES (rounds toward zero), like Haskell's quot and C's /. Same as div
332 for positives; differs by one for negatives.
333 >>> quot(7, 2), quot(-7, 2)
334 (3, -3)
335
336 rem -- the remainder that matches quot: its sign follows the DIVIDEND.
337 >>> rem(7, 2), rem(-7, 2), rem(7, -2)
338 (1, -1, 1)
339
340 quotRem -- truncating quotient and remainder at once.
341 >>> quotRem(-7, 2)
342 (-3, -1)
343
344 gcd -- greatest common divisor by Euclid's recursion, always >= 0 like Haskell's. Recursion is safe here:
345 the depth is at most ~90 even for 64-bit inputs (worst case: consecutive Fibonacci).
346 >>> gcd(48, 18)
347 6
348 >>> gcd(-4, 6), gcd(17, 5)
349 (2, 1)
350
351 lcm -- least common multiple via gcd, also always >= 0; defined as 0 when either input is 0.
352 >>> lcm(4, 6)

```

```

353 12
354 >>> lcm(-4, 6), lcm(0, 5)
355 (12, 0)
356
357 hypot -- the Euclidean distance sqrt(x^2 + y^2). Note it returns a float.
358 >>> hypot(3, 4)
359 5.0
360
361 isqrt -- floor square root using Newton's method on INTEGERS -- no floats, so no precision loss on huge
362 numbers; a negative input raises rather than lying.
363 >>> isqrt(16), isqrt(17)
364 (4, 4)
365 >>> isqrt(10**18)
366 1000000000
367
368 Roll your own: Newton's iteration on integers -- repeatedly replace x with the average of x and n // x.
369 It converges from above, so loop while the new estimate still shrinks; all arithmetic is //, so no float
370 (and no float rounding lie) ever appears.
371
372 ===== 4. list basics =====
373
374 head -- the first element. A PARTIAL function: on an empty list it raises, so guard with null() or reach
375 for find/fromMaybe when emptiness is possible.
376 >>> head([1, 2, 3])
377 1
378 >>> head([])
379 Traceback (most recent call last):
380 ...
381 IndexError: list index out of range
382
383 tail -- everything after the first element. head and tail together are the grammar of structural
384 recursion: do something with head, recurse on tail.
385 >>> tail([1, 2, 3])
386 [2, 3]
387 >>> tail([1])
388 []
389
390 init -- everything before the last element (the mirror of tail).
391 >>> init([1, 2, 3])
392 [1, 2]
393
394 last -- the final element. Partial, like head.
395 >>> last([1, 2, 3])
396 3
397
398 null -- is the list empty? The guard that makes head/tail safe to use.
399 >>> null([]), null([1])
400 (True, False)
401
402 elem -- membership test, argument order (needle, haystack).
403 >>> elem(2, [1, 2, 3])
404 True
405 >>> elem('q', "abc")
406 False
407
408 notElem -- non-membership.
409 >>> notElem(9, [1, 2, 3])
410 True
411
412 replicate -- n copies of one value, as a list.
413 >>> replicate(3, 'x')
414 ['x', 'x', 'x']
415 >>> replicate(0, 'x')
416 []
417
418 drop -- discard the first n elements, keep the rest; finite lists only (it materialises the input), and a
419 negative n drops nothing.
420 >>> drop(2, [1, 2, 3, 4])
421 [3, 4]
422 >>> drop(10, [1, 2]), drop(-3, [1, 2])
423 ([], [1, 2])
424
425 splitAt -- cut a list into (first n, rest) in one call; a negative n splits at the front. The pieces glue
426 back with +, which is why rotations and chunking fall out of it.
427 >>> splitAt(2, [1, 2, 3, 4])
428 ([1, 2], [3, 4])
429 >>> splitAt(0, [1, 2]), splitAt(-2, [1, 2])
430 ([], [1, 2]), ([], [1, 2])
431
432 nub -- deduplicate, FIRST occurrence wins, order preserved. The trick: dict.fromkeys builds a dict (whose
433 keys are unique and insertion-ordered) and throws away the values. Elements must be hashable. Compare
434 set(xs), which destroys order.
435 >>> nub([3, 1, 3, 2, 1])
436 [3, 1, 2]
437 >>> nub("mississippi")
438 ['m', 'i', 's', 'p']
439
440 lookup -- the first value paired with a key in an association list (a list of (key, value) tuples), or

```

```

441 None if absent. First match wins, so an assoc list can shadow like a scope chain.
442 >>> lookup('b', [('a', 1), ('b', 2)])
443 2
444 >>> lookup('a', [('a', 1), ('a', 99)])
445 1
446 >>> lookup('z', [('a', 1)]) is None
447 True
448
449 zip_ -- pair two iterables positionally, stopping at the shorter; returns a real list (the builtin zip
450 returns a lazy iterator), and either side may be any iterable, streams included.
451 >>> zip_([1, 2], [10, 20, 30])
452 [(1, 10), (2, 20)]
453 >>> zip_("ab", count(0))
454 [('a', 0), ('b', 1)]
455
456 zip3 -- the three-list version.
457 >>> zip3([1, 2], ['a', 'b'], [True, False])
458 [(1, 'a', True), (2, 'b', False)]
459
460 unzip -- a list of pairs back into a pair of lists; the inverse of zip_.
461 >>> unzip([(1, 'a'), (2, 'b')])
462 ([1, 2], ['a', 'b'])
463 >>> unzip([])
464 ([], [])
465
466 transpose -- rows become columns, RAGGED-SAFE exactly like Haskell's: short rows simply drop out of later
467 columns; nothing is silently truncated.
468 >>> transpose([[1, 2, 3], [4, 5, 6]])
469 [[1, 4], [2, 5], [3, 6]]
470 >>> transpose([[1, 2, 3], [4, 5]])
471 [[1, 4], [2, 5], [3]]
472 >>> transpose([])
473 []
474
475 Rows may be any iterables, even one-shot ones -- the rows are materialised first, so a generator of
476 iterators transposes like a list of lists (without that step the generator is spent by the length scan):
477 >>> transpose(iter([1, 2]) for _ in range(2))
478 [[1, 1], [2, 2]]
479 >>> transpose(["abc", "de"])
480 [['a', 'd'], ['b', 'e'], ['c']]
481
482 Roll your own: materialise the rows, find the longest, then for each column index keep xs[i] only from
483 rows that reach it -- the 'if i < len(xs)' filter is precisely where short rows drop out of later columns
484 instead of truncating everyone.
485
486 enum -- pair every element with its index: a list-returning enumerate. The shape that feeds index-aware
487 folds (two_sum's seen-dict, next_greater's stack).
488 >>> enum(['a', 'b'])
489 [(0, 'a'), (1, 'b')]
490 >>> enum(['a', 'b'], start=1)
491 [(1, 'a'), (2, 'b')]
492
493 pairwise -- every element with its RIGHT neighbour: zip xs (tail xs). The tool for any rule about
494 adjacency -- deltas, local comparisons, Roman numeral subtraction.
495 >>> pairwise([1, 4, 9])
496 [(1, 4), (4, 9)]
497 >>> map_(lambda p: p[1] - p[0], pairwise([3, 10, 6]))
498 [7, -4]
499
500 isPrefixOf -- does xs start with p? Works on strings and lists alike (both are normalised through
501 list()).
502 >>> isPrefixOf("re", "rebuild")
503 True
504 >>> isPrefixOf([1, 2], [1, 2, 3])
505 True
506 >>> isPrefixOf("", "anything")
507 True
508
509 isSuffixOf -- does xs end with p? Implemented by slicing from len(xs) - len(p), NOT by xs[-len(p):],
510 because when p is empty that slice is xs[-0:] == the WHOLE list. A scar worth keeping visible.
511 >>> isSuffixOf("ing", "folding")
512 True
513 >>> isSuffixOf("", "x")
514 True
515 >>> isSuffixOf("abc", "bc")
516 False
517
518 stripPrefix -- Just the rest after the prefix, or Nothing: returns xs[len(p):] when the prefix matches,
519 None when it does not. Pairs with fromMaybe and mapMaybe.
520 >>> stripPrefix("re", "rebuild")
521 'build'
522 >>> stripPrefix("un", "rebuild") is None
523 True
524 >>> mapMaybe(partial(stripPrefix, "err:"), ["err:a", "ok:b", "err:c"])
525 ['a', 'c']
526
527 ===== 5. higher-order lists =====
528

```

```

529 map_ -- apply a function to every element; returns a real list, eagerly (the builtin map is lazy). The
530 workhorse transformation.
531 >>> map_(lambda x: x * x, [1, 2, 3])
532 [1, 4, 9]
533 >>> map_(str.upper, ["ab", "cd"])
534 ['AB', 'CD']
535
536 filter_ -- keep the elements satisfying a predicate. Scans the WHOLE list; if you only want the first
537 hit, that is find.
538 >>> filter_(even, [1, 2, 3, 4])
539 [2, 4]
540 >>> filter_(str.isdigit, "alb22")
541 ['1', '2', '2']
542
543 find -- the first element satisfying the predicate, else None. Lazy: it stops at the first hit, which
544 folds cannot do, and therefore works even on infinite streams. Finish with fromMaybe if you need a
545 default.
546 >>> find(even, [1, 3, 4, 6])
547 4
548 >>> find(even, [1, 3]) is None
549 True
550 >>> find(lambda x: x % 7 == 0, count(1))
551 7
552
553 How it works: find wraps a generator expression in next() with a None default. The generator is the point
554 -- elements are tested one at a time, on demand, and the search STOPS at the first hit. Nothing after the
555 match is ever examined.
556
557 Why it earns its place: folds and filter_ consume the whole list; they cannot stop early. find can --
558 which makes it the right tool the moment the word "first" appears in a statement, and the only tool that
559 survives an infinite stream. Land defaults with fromMaybe.
560 >>> find(lambda x: x * x > 50, count(1))
561 8
562 >>> fromMaybe('?', find(str.isdigit, "abc7de"))
563 '?'
564
565 Versus filter_: filter_ answers "which ones?"; find answers "who was first, if anyone?". Taking
566 filter_(p, xs)[0] instead does wasted work and explodes when nothing matches; find does neither.
567
568 Remember it as: find is the early exit that folds cannot perform.
569
570 Roll your own: a generator expression handed to next() with a default. The generator is what makes it
571 lazy -- elements are produced one at a time, so the first hit stops everything -- and the default is what
572 makes it total.
573
574 partition -- split into (satisfy, don't) in ONE pass, both sides in original order; the predicate runs
575 exactly once per element, so it may be expensive or even stateful.
576 >>> partition(even, [1, 2, 3, 4])
577 ([2, 4], [1, 3])
578 >>> partition(str.isalpha, "alb2")
579 (['a', 'b'], ['1', '2'])
580
581 Roll your own: two buckets and one conditional append -- (yes if pred(x) else no).append(x). The
582 conditional expression picks the LIST, then the append mutates it; one pass, exactly one predicate call
583 per element.
584
585 mapMaybe -- map a function that returns a value or None, and keep only the values: parse-and-filter in
586 ONE pass. The shape for messy input -- write a parser that answers None for garbage, then mapMaybe it
587 over everything. dict.get is already such a function.
588 >>> mapMaybe(lambda s: int(s) if s.isdigit() else None, ["3", "x", "7"])
589 [3, 7]
590 >>> mapMaybe({'a': 1, 'b': 2}.get, ['a', 'x', 'b'])
591 [1, 2]
592
593 How it works: run f over every element; f answers a VALUE when it can make sense of the input and None
594 when it cannot; keep only the values. Map and filter fused into one pass, with the parser itself deciding
595 what survives.
596 >>> parse_int = lambda s: int(s) if s.lstrip('-').isdigit() else None
597 >>> mapMaybe(parse_int, ["12", "x", "-3", ""])
598 [12, -3]
599
600 Why it earns its place: messy input. The alternative is a loop with an if, a flag and an append -- three
601 chances for a bug. Here the contract is crisp: write parse(item) -> value-or-None once, and mapMaybe
602 handles every item. dict.get is already such a function, so lookups that may miss compose directly.
603 >>> legend = {'a': 1, 'b': 2}
604 >>> mapMaybe(legend.get, "cabbage")
605 [1, 2, 2, 1]
606
607 The Haskell connection: Data.Maybe's mapMaybe, with None standing in for Nothing -- the "parse, don't
608 validate" discipline: turn questionable data into clean data at the boundary, and everything downstream
609 stops worrying.
610
611 Remember it as: map with a parser, drop the Nothings, one pass.
612
613 catMaybes -- keep the non-None values; mapMaybe with the identity.
614 >>> catMaybes([1, None, 2, None])
615 [1, 2]
616

```

```

617 concat -- flatten ONE level of nesting.
618 >>> concat([[1, 2], [3], []])
619 [1, 2, 3]
620 >>> concat(["ab", "cd"])
621 ['a', 'b', 'c', 'd']
622
623 concatMap -- map each element to a LIST, and pool all the results: concat after map_, fused. The backbone
624 of enumeration -- "for every choice, produce all outcomes".
625 >>> concatMap(lambda w: [w, w.upper()], ["a", "b"])
626 ['a', 'A', 'b', 'B']
627 >>> concatMap(lambda x: [x] * x, [1, 2, 3])
628 [1, 2, 2, 3, 3, 3]
629 >>> concatMap(lambda x: [x, -x], [1, 2]) == concat(map_(lambda x: [x, -x], [1, 2]))
630 True
631
632 starmap -- map a MULTI-argument function over a list of argument tuples: map_ composed with uncurry.
633 >>> starmap(lambda a, b: a * b, [(2, 3), (4, 5)])
634 [6, 20]
635 >>> starmap(pow, [(2, 3), (3, 2)])
636 [8, 9]
637
638 zipWith -- combine two lists elementwise with a function; stops at the shorter. zip_ is just
639 zipWith(make-a-pair).
640 >>> zipWith(add, [1, 2], [10, 20])
641 [11, 22]
642 >>> zipWith(lambda a, b: a > b, [3, 1], [2, 5, 9])
643 [True, False]
644
645 zipWith3 -- the three-list version.
646 >>> zipWith3(lambda a, b, c: a + b + c, [1], [2], [3])
647 [6]
648
649 cross -- every pairing of two lists (the cartesian product). For the product of a list with ITSELF n
650 times, see replicateM.
651 >>> cross([1, 2], "ab")
652 [(1, 'a'), (1, 'b'), (2, 'a'), (2, 'b')]
653 >>> len(cross(range(3), range(4)))
654 12
655
656 ===== 6. folds =====
657
658 foldl -- THE left fold, and the prelude's most important definition. It is exactly this loop: acc = init;
659 for x in xs: acc = f(acc, x); return acc. The combiner takes (ACCUMULATOR, element) -- accumulator first.
660 Argument order of foldl itself: (f, xs, init), with init optional -- omitted, the first element seeds the
661 accumulator (and an empty list is then an error).
662 >>> foldl(lambda acc, x: acc * 10 + x, [1, 2, 3], 0)
663 123
664 >>> foldl(max, [3, 9, 2]) # no init: 3 seeds it
665 9
666 >>> foldl(add, [], 0) # empty + init: the init comes back
667 0
668 >>> foldl(lambda acc, w: acc + len(w), ["ab", "c"], 0)
669 3
670
671 How it works: foldl is precisely this loop, given a name and a contract:
672 >>> def foldl_spelled_out(f, xs, init):
673 ...     acc = init
674 ...     for x in xs:
675 ...         acc = f(acc, x)
676 ...     return acc
677 >>> foldl_spelled_out(add, [1, 2, 3], 0) == foldl(add, [1, 2, 3], 0)
678 True
679
680 Two disciplines hide in the signature. The combiner takes (ACCUMULATOR, element) -- accumulator first,
681 always; reversing them is the classic fold bug. And the optional init is guarded by NOTHING, so the fold
682 can tell "no seed -- use the first element" from "the seed is None".
683
684 Why it earns its place: the fold is the universal list consumer. sum, max, reverse, "build a dict", "run
685 a state machine over events" -- each is one choice of combiner and seed. When you can say "carry THIS
686 state, update it THIS way per element", the loop is already written.
687 >>> foldl(lambda acc, x: [x] + acc, [1, 2, 3], []) # reverse is a fold
688 [3, 2, 1]
689 >>> foldl(lambda acc, w: acc + len(w), ["ab", "cde"], 0)
690 5
691
692 Remember it as: a fold is a for-loop whose state has been given a contract.
693
694 Roll your own: seed the accumulator -- init if one was given, else the first element (the NOTHING guard
695 is what lets None be a legitimate seed) -- then a single for-loop folding acc = f(acc, x). The entire
696 contract is which side the accumulator sits on.
697
698 foldl1 -- foldl seeded by the first element, spelled out. Use when the accumulator has the same type as
699 the elements and emptiness cannot happen.
700 >>> foldl1(add, [1, 2, 3])
701 6
702 >>> foldl1(min, [3, 1, 2])
703 1
704

```

```

705 foldr -- the RIGHT fold: nests from the right, f takes (element, ACCUMULATOR) -- element first, mirror of
706 foldl. Implemented as foldl of the flipped function over the reversed list: iterative and stack-safe, an
707 identity that only holds because Python is strict. The parenthesisat example makes the direction
708 visible.
709 >>> foldr(lambda x, acc: '(' + x + '+' + acc + ')', "abc", "z")
710 '(a+(b+(c+z)))'
711 >>> foldl(lambda acc, x: '(' + acc + '+' + x + ')', "abc", "z")
712 '(((z+a)+b)+c)'
713 >>> foldr(lambda x, acc: [x] + acc, [1, 2, 3], [])
714 [1, 2, 3]
715
716 How it works: the mirror image -- the combiner takes (element, ACCUMULATOR) and the nesting grows from
717 the RIGHT: f(a, f(b, f(c, z))). The prelude does not write a second recursion: foldr = foldl(flip(f),
718 reversed(xs), init). Flip fixes the argument order, reversing fixes the traversal, and the existing
719 engine runs backwards.
720
721 The honesty clause: that identity only holds because Python is STRICT. Haskell's foldr can process
722 infinite lists -- laziness lets f decide whether the rest is ever needed; ours materialises and reverses
723 the list first, so it cannot. Same name, same finite behaviour, different superpower.
724
725 Why it earns its place: building right-to-left. Consing onto a front is natural from the right -- section
726 13's from_list IS a foldr of ListNode, and here it is, derived by hand:
727 >>> foldr(lambda x, acc: [x] + acc, [1, 2, 3], []) # rebuild, order kept
728 [1, 2, 3]
729 >>> to_list(foldr(ListNode, [1, 2, 3], None)) # from_list, spelled out
730 [1, 2, 3]
731
732 Remember it as: foldl eats left-to-right; foldr thinks right-to-left -- and in strict Python it is foldl
733 wearing flip and a reversal.
734
735 foldr1 -- right fold seeded by the LAST element.
736 >>> foldr1(lambda x, acc: x - acc, [1, 2, 3]) # 1 - (2 - 3)
737 2
738
739 product -- multiply everything: the numeric fold, seeded with 1 so an empty product is 1.
740 >>> product([2, 3, 4])
741 24
742 >>> product([])
743 1
744
745 compose -- fold any number of functions into one; the RIGHTMOST runs first, like mathematical composition
746 f(g(x)).
747 >>> compose(str.strip, str.upper)(" hi ")
748 'HI'
749 >>> f = compose(succ, succ, succ)
750 >>> f(0)
751 3
752
753 How it works: a fold over the functions themselves -- neighbours merge into lambda x: f(g(x)), seeded
754 with the identity, so compose() with no arguments is id. The RIGHTMOST function runs first, matching
755 mathematical (f . g)(x) = f(g(x)).
756 >>> compose()(42) == id(42)
757 True
758 >>> compose(len, str.strip)(" hi ") # strip first, then len
759 2
760
761 Why it earns its place: it turns a pipeline into a VALUE. A composed function can be named, mapped,
762 stored in a rules table -- processing steps become data you can shuffle. The constraint is the one curry
763 answers: every stage must be unary, which is why compose-heavy code leans on curry and partial to shape
764 its stages.
765 >>> normalize = compose(str.lower, str.strip)
766 >>> map_(normalize, [" Hi ", " YO "])
767 ['hi', 'yo']
768
769 Remember it as: foldr (.) id -- a pipeline you can hold in your hand.
770
771 Roll your own: it IS foldr (.) id -- foldr over the functions with x as the seed, each stage applied to
772 the accumulated result: foldr(lambda f, acc: f(acc), fns, x). Because the prelude's foldr is an iterative
773 loop underneath, stack depth stays constant no matter how many stages you compose.
774
775 subsequences -- every subset of the elements (the powerset), built by a fold in which each element
776 DOUBLES the accumulator: keep every existing subset, and every existing subset extended by the newcomer.
777 2^n results -- a brute-force license for n up to about 20, and you should say that bound out loud before
778 using it.
779 >>> subsequences([1, 2])
780 [[], [1], [2], [1, 2]]
781 >>> subsequences("ab")
782 [[], ['a'], ['b'], ['a', 'b']]
783 >>> len(subsequences("abcd"))
784 16
785
786 How it works: watch the accumulator double. Start from [[]] -- one subset, the empty one. Each element x
787 rewrites acc as acc + [s + [x] for s in acc]: every existing subset survives (x left out), and every
788 existing subset reappears extended by x (x taken). n elements, n doublings, 2^n subsets, in a stable
789 order.
790 >>> acc = [[]]
791 >>> acc = acc + [s + [1] for s in acc]; acc
792 [[], [1]]

```

```

793 >>> acc = acc + [s + [2] for s in acc]; acc
794 [[], [1], [2], [1, 2]]
795
796 Why it earns its place: it is a brute-force LICENSE. At n <= 20, checking every subset is a few million
797 cheap operations -- honest, exhaustive, and unbeatable to debug. Say the bound out loud ("2^15 is 32768
798 -- fine") and enumerate; cleverness can come later if the bound grows.
799 >>> best = maxOn(sum, [s for s in subsequences([3, -1, 4, -2]) if sum(s) <= 6])
800 >>> sum(best)
801 6
802
803 Remember it as: the powerset fold -- each element doubles the world.
804
805 replicateM -- every length-n word over an alphabet: cross applied n times, |xs|^n results. The other
806 brute-force license -- all bitstrings, all dice rolls, all assignments of n slots.
807 >>> replicateM(2, "ab")
808 [['a', 'a'], ['a', 'b'], ['b', 'a'], ['b', 'b']]
809 >>> len(replicateM(3, [0, 1]))
810 8
811
812 ===== 7. scans =====
813
814 accumulate -- the generator behind the scans: yields the running combination of everything seen so far.
815 With no function it sums; 'initial' seeds it. This is Python's itertools name with Haskell's scan
816 semantics.
817 >>> list(accumulate([1, 2, 3]))
818 [1, 3, 6]
819 >>> list(accumulate([1, 2], initial=10))
820 [10, 11, 13]
821 >>> list(accumulate([3, 1, 2], min))
822 [3, 1, 1]
823
824 Roll your own: the same skeleton as foldl, but a GENERATOR -- yield the accumulator once before the loop,
825 then yield again after every combine. The early bare return on an empty, seedless input is what makes an
826 empty scan come back empty instead of raising.
827
828 scanl -- a fold that KEEPS ITS HISTORY: returns every intermediate accumulator, starting with the seed,
829 so the result has len(xs)+1 elements and its last element equals the foldl. Argument order (f, z, xs) --
830 seed in the middle, Haskell's order, unlike foldl's trailing init. Prefix sums are the canonical scan.
831 >>> scanl(add, 0, [3, 1, 4])
832 [0, 3, 4, 8]
833 >>> last(scanl(add, 0, [1, 2, 3])) == foldl(add, [1, 2, 3], 0)
834 True
835
836 The DP secret: many celebrated one-pass algorithms are scans in a trench coat, because a scan IS a
837 dynamic-programming table -- entry i holds the answer for the prefix ending at i. Kadane's famous
838 maximum-subarray "trick" is one scanl and a max:
839 >>> xs = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
840 >>> max(scanl(lambda best, x: max(x, best + x), xs))
841 6
842
843 Prefix sums, the other superpower: scan once with (+) and every range sum becomes two reads, pref[j] -
844 pref[i] -- O(n) window questions collapse to O(1).
845 >>> pref = scanl(add, 0, [3, 1, 4, 1, 5])
846 >>> pref[4] - pref[1] # sum of elements 1..3
847 6
848
849 Remember it as: when you need the state at EVERY step, one scan replaces n folds.
850
851 scanl1 -- the history-keeping fold seeded by the first element; len(xs) results. Running maxima and
852 minima live here.
853 >>> scanl1(min, [3, 1, 2])
854 [3, 1, 1]
855 >>> scanl1(max, [1, 3, 2, 5])
856 [1, 3, 3, 5]
857
858 scanr -- the suffix scan: folds from the RIGHT, showing every suffix's result; the first element is the
859 full fold, the last is the seed. Suffix sums for product_except_self-style problems.
860 >>> scanr(add, 0, [1, 2, 3])
861 [6, 5, 3, 0]
862
863 scanr1 -- suffix scan seeded by the last element.
864 >>> scanr1(add, [1, 2, 3])
865 [6, 5, 3]
866
867 ===== 8. streams =====
868
869 count -- the infinite arithmetic sequence start, start+step, ... Never consume it whole: pair it with
870 take, takewhile or find, which stop.
871 >>> take(3, count(10))
872 [10, 11, 12]
873 >>> take(3, count(0, 5))
874 [0, 5, 10]
875
876 repeat -- one value forever, or exactly n times with the second argument.
877 >>> take(2, repeat('x'))
878 ['x', 'x']
879 >>> list(repeat('x', 3))
880 ['x', 'x', 'x']

```

```

881 >>> list(repeat('x', 0))
882 []
883
884 cycle -- a finite list looped forever; an EMPTY list raises immediately (Haskell errors too -- the
885 alternative is a silent infinite hang).
886 >>> take(5, cycle([1, 2]))
887 [1, 2, 1, 2, 1]
888
889 iterate -- x, f(x), f(f(x)), ... forever: repeated application as a stream. Any deterministic state
890 machine becomes iterate on its step function -- Fibonacci is iterate on a pair.
891 >>> take(4, iterate(lambda x: 2 * x, 1))
892 [1, 2, 4, 8]
893 >>> step = lambda p: (snd(p), fst(p) + snd(p))
894 >>> map_(fst, take(6, iterate(step, (0, 1))))
895 [0, 1, 1, 2, 3, 5]
896
897 unfoldr -- iterate's FINITE twin, and the cure for threading (state, acc) tuples through loops. You
898 supply f(seed) returning either (value, seed') to emit a value and continue, or None to stop; unfoldr
899 collects the values. Anything that "peels" a structure -- digits, chunks, parent chains -- is an unfoldr.
900 >>> unfoldr(lambda m: None if m == 0 else (m % 10, m // 10), 407)
901 [7, 0, 4]
902 >>> unfoldr(lambda n: None if n == 0 else (n, n - 1), 5)
903 [5, 4, 3, 2, 1]
904 >>> unfoldr(const(None), "seed")          # stop immediately: empty
905 []
906
907 How it works: the step function carries a two-outcome contract -- f(seed) returns (value, next_seed) to
908 emit and continue, or None to stop -- and unfoldr collects what was emitted. It is the fold REVERSED: a
909 fold consumes a list into a summary; an unfold grows a list out of a seed. (Category theory says
910 anamorphism; interviewers say "generate the sequence".)
911
912 Why it earns its place: it retires the (state, accumulator) tuple you would otherwise thread through a
913 while loop. Anything that PEELS -- digits off a number, chunks off a list, parents up a tree, a Collatz
914 trajectory -- states its entire logic in the step function.
915 >>> unfoldr(lambda n: None if n == 1 else (n, n // 2 if even(n) else 3 * n + 1), 6)
916 [6, 3, 10, 5, 16, 8, 4, 2]
917
918 Versus iterate: iterate is the infinite cousin -- an unfold whose step never says None, consumed with
919 take. Reach for iterate when the stream is conceptually endless, unfoldr when the seed itself knows where
920 the end is.
921
922 Remember it as: fold tears down; unfold builds up; the seed carries the future.
923
924 More unfolds, same two-outcome contract -- peeling chunks off a list, bits off a number, walking a parent
925 chain to its root:
926 >>> unfoldr(lambda xs: None if xs == [] else (xs[:2], xs[2:]), [1, 2, 3, 4, 5])
927 [[1, 2], [3, 4], [5]]
928 >>> unfoldr(lambda m: None if m == 0 else (m % 2, m // 2), 13)          # bits, LSB first
929 [1, 0, 1, 1]
930 >>> parents = {'c': 'b', 'b': 'a', 'a': None}
931 >>> unfoldr(lambda n: None if n is None else (n, parents[n]), 'c')      # ancestry
932 ['c', 'b', 'a']
933
934 And the seed can carry several facts at once -- here (current, next, remaining) generates Fibonacci with
935 its own countdown built in:
936 >>> unfoldr(lambda s: None if s[2] == 0 else (s[0], (s[1], s[0] + s[1], s[2] - 1)), (0, 1, 8))
937 [0, 1, 1, 2, 3, 5, 8, 13]
938
939 Roll your own: prime the pump with step = f(seed), then loop while step is not None -- append the value,
940 feed the new seed back to f. The two-outcome contract lives entirely in f; unfoldr is just the loop that
941 honours it, plus the list that collects what was emitted.
942
943 chain -- concatenate iterables lazily, one after another: a generator expression, so the tail is never
944 touched until asked for (chain over count() is fine). concat is the strict, list-building cousin.
945 >>> list(chain([1], [2, 3]))
946 [1, 2, 3]
947 >>> take(4, chain([1, 2], count(10)))
948 [1, 2, 10, 11]
949
950 ===== 9. slicing & spans =====
951
952 islice -- at most the first n items of ANY iterable, lazily; the safe window onto an infinite stream.
953 Runs short without complaint.
954 >>> list(islice(count(0), 3))
955 [0, 1, 2]
956 >>> list(islice([1, 2], 5))
957 [1, 2]
958
959 take -- the first n, as a real list. take/drop are the positional pair; takeWhile/dropWhile are the
960 conditional pair.
961 >>> take(3, count(1))
962 [1, 2, 3]
963 >>> take(2, [7, 8, 9])
964 [7, 8]
965
966 takewhile -- yield elements WHILE the predicate holds, then stop at the first failure (a generator).
967 Contrast filter_: filter_ scans everything; takewhile stops -- on [2, 4, 5, 6], filter_ keeps the 6,
968 takewhile does not.

```

```

969 >>> list(takewhile(even, [2, 4, 5, 6]))
970 [2, 4]
971 >>> filter_(even, [2, 4, 5, 6])
972 [2, 4, 6]
973
974 takeuntil -- like takewhile(not p), but INCLUDES the first element where p holds -- the do-while flavour:
975 "read up to and including the terminator".
976 >>> list(takeuntil(lambda x: x == 3, [1, 2, 3, 4]))
977 [1, 2, 3]
978 >>> list(takeuntil(lambda x: x > 0, [5, 6]))
979 [5]
980
981 dropwhile -- discard while the predicate holds, then yield EVERYTHING after the first failure, no more
982 testing. takewhile's complement.
983 >>> list(dropwhile(lambda x: x < 3, [1, 2, 3, 1]))
984 [3, 1]
985
986 takeWhile -- takewhile, delivered as a list (capital W = list version).
987 >>> takeWhile(lambda x: x < 3, [1, 2, 3, 1])
988 [1, 2]
989
990 dropWhile -- dropwhile, delivered as a list.
991 >>> dropWhile(lambda x: x < 3, [1, 2, 3, 1])
992 [3, 1]
993
994 span -- split at the first failure: (longest satisfying prefix, the rest), computed in ONE pass so it is
995 safe on one-shot iterators (the old two-scan version silently broke on generators -- this is the fix). A
996 lexer in one call.
997 >>> span(lambda x: x < 3, [1, 2, 3, 4, 1])
998 ([1, 2], [3, 4, 1])
999 >>> span(str.isdigit, "123abc")
1000 ([ '1', '2', '3'], [ 'a', 'b', 'c'])
1001
1002 Roll your own: materialise xs once (that is the one-pass guarantee), locate the first failing index with
1003 next() over enumerate -- defaulting to len(xs) when the predicate never fails -- and slice twice. The
1004 previous two-scan version ran takewhile and dropwhile separately and silently broke on one-shot
1005 generators; this is the repair.
1006
1007 break_ -- span with the predicate negated: split where the condition first HOLDS.
1008 >>> break_(lambda x: x > 2, [1, 2, 3, 1])
1009 ([1, 2], [3, 1])
1010
1011 inits -- every prefix, from empty to the whole thing; len(xs)+1 of them.
1012 >>> inits([1, 2])
1013 [[], [1], [1, 2]]
1014 >>> inits("ab")
1015 ['', 'a', 'ab']
1016
1017 tails -- every suffix, whole thing first, down to empty. Substring problems start here: every substring
1018 is a prefix of some suffix.
1019 >>> tails("abc")
1020 ['abc', 'bc', 'c', '']
1021 >>> tails([1, 2])
1022 [[1, 2], [2], []]
1023
1024 chunksOf -- consecutive pieces of length n; the last may be short. Works on lists and strings alike since
1025 it is pure slicing.
1026 >>> chunksOf(2, [1, 2, 3, 4, 5])
1027 [[1, 2], [3, 4], [5]]
1028 >>> chunksOf(3, "abcdefg")
1029 ['abc', 'def', 'g']
1030
1031 ===== 10. sorting & searching =====
1032
1033 sortOn -- sort by a key function, evaluated once per element. TUPLE KEYS are the power move: (primary,
1034 secondary) sorts two levels at once, and negating a numeric component flips just that level -- (-count,
1035 word) is "count descending, ties alphabetical", which reverse=True cannot express.
1036 >>> sortOn(len, ["bbb", "a", "cc"])
1037 ['a', 'cc', 'bbb']
1038 >>> sortOn(lambda kv: (-kv[1], kv[0]), [('b', 2), ('a', 2), ('c', 3)])
1039 [('c', 3), ('a', 2), ('b', 2)]
1040
1041 minOn -- the element with the smallest key, in O(n). sortOn + head gives the same answer for O(n log n)
1042 -- pay the sort only when you need ALL of them ordered.
1043 >>> minOn(abs, [-3, 2, 5])
1044 2
1045 >>> minOn(snd, [('a', 9), ('b', 1)])
1046 ('b', 1)
1047
1048 maxOn -- the element with the largest key, O(n); the FIRST winner on ties, so pre-sorting the input
1049 controls the tie-break.
1050 >>> maxOn(len, ["aa", "b", "cc"])
1051 'aa'
1052
1053 cmp_to_key -- adapts an old-style comparator (returning negative, zero or positive) into the key=
1054 protocol. Reach for it only when a rule genuinely compares two elements against each other and no per-
1055 element key exists.
1056 >>> sorted(["bb", "a"], key=cmp_to_key(lambda a, b: len(a) - len(b)))

```

```

1057 ['a', 'bb']
1058
1059 Roll your own: a tiny wrapper class implementing ONE dunder -- __lt__ defers to cmp(a, b) < 0. Python's
1060 sort only ever asks "is a less than b?", so that single method is enough to smuggle any comparator
1061 through the key= interface.
1062
1063 sortBy -- comparator sort, Python 2 style, built on cmp_to_key.
1064 >>> sortBy(lambda a, b: b - a, [1, 3, 2])
1065 [3, 2, 1]
1066
1067 merge -- join two ALREADY-SORTED lists into one sorted list, stably, in O(n+m): the two-pointer loop at
1068 the heart of merge sort.
1069 >>> merge([1, 4, 6], [2, 3, 7])
1070 [1, 2, 3, 4, 6, 7]
1071 >>> merge([], [1, 2])
1072 [1, 2]
1073
1074 Roll your own: two cursors, always copying the smaller head forward; the <= (rather than <) is what makes
1075 the merge STABLE, preferring the left list on ties. When either side runs dry, glue both tails on -- one
1076 of them is empty, so the + is harmless.
1077
1078 bisect_left -- binary search over a SORTED list: the first index whose element is >= x; equivalently,
1079 where x would insert to the left of its equals. If a[i] != x there, x is absent -- that check is binary
1080 search.
1081 >>> bisect_left([10, 20, 20, 30], 20)
1082 1
1083 >>> bisect_left([10, 20, 30], 25)           # insertion point for a missing value
1084 2
1085
1086 How it works: on a sorted list the predicate "element < x" reads True for a prefix and False for the rest
1087 -- TTTTFFFF. Binary search does not hunt for x; it finds that BOUNDARY, halving the interval per probe.
1088 bisect_left returns the first index >= x, bisect_right the first > x, and from those two facts three
1089 idioms fall out.
1090 >>> xs = [10, 20, 20, 30]
1091 >>> bisect_left(xs, 20), bisect_right(xs, 20)      # the run of 20s occupies [1, 3)
1092 (1, 3)
1093
1094 Idiom one, membership: x is present iff the slot bisect_left names actually holds it. Idiom two,
1095 insertion point: that same index is where x belongs to keep the list sorted. Idiom three, counting: right
1096 minus left counts occurrences -- and with two different probes, the values inside any range.
1097 >>> i = bisect_left(xs, 25)
1098 >>> i, (i < len(xs) and xs[i] == 25)              # 25 would insert at 3; absent
1099 (3, False)
1100
1101 Why it earns its place: every threshold ladder is secretly a sorted list. Grading bands, tax brackets,
1102 rate tiers -- one bisect replaces the if/elif staircase:
1103 >>> grade = lambda score: "FDCBA"[bisect_right([60, 70, 80, 90], score)]
1104 >>> grade(59), grade(60), grade(95)
1105 ('F', 'D', 'A')
1106
1107 Remember it as: bisect finds the boundary in a sea of Trues and Falses; everything else is reading that
1108 boundary three ways.
1109
1110 Roll your own: the lo/hi halving loop over an invariant -- everything left of lo is < x, everything from
1111 hi rightwards is >= x. One comparison, a[mid] < x, decides which half survives; when lo meets hi, that
1112 meeting point IS the boundary. bisect_right is the identical loop with <= in place of < -- one character
1113 moves the boundary to the far side of the equals.
1114
1115 bisect_right -- the first index whose element is > x: where x would insert to the RIGHT of its equals.
1116 right minus left counts the occurrences.
1117 >>> bisect_right([10, 20, 20, 30], 20)
1118 3
1119 >>> xs = [10, 20, 20, 30]
1120 >>> bisect_right(xs, 20) - bisect_left(xs, 20)
1121 2
1122
1123 ===== 11. strings =====
1124
1125 words -- split on any whitespace; runs collapse, edges vanish. The inverse direction is unwords, and the
1126 round trip normalises spacing.
1127 >>> words("  such  spacing ")
1128 ['such', 'spacing']
1129 >>> unwords(words("  a  b "))
1130 'a b'
1131
1132 unwords -- join words with single spaces.
1133 >>> unwords(['a', 'b', 'c'])
1134 'a b c'
1135
1136 lines -- split a string into its lines (no trailing newlines kept).
1137 >>> lines("one\ntwo")
1138 ['one', 'two']
1139
1140 unlines -- join lines, TERMINATING each with a newline (not separating -- note the trailing one).
1141 >>> unlines(['a', 'b'])
1142 'a\nb\n'
1143
1144 ===== 12. containers =====

```

```

1145
1146 defaultdict -- a dict whose missing keys create themselves via a factory: touch d[k] and factory() is
1147 installed there. This "autovivification" makes grouping and counting one-liners. Gotcha: READING a
1148 missing key also plants it -- use 'in' or .get when you only want to look.
1149 >>> d = defaultdict(list)
1150 >>> d['x'].append(1)
1151 >>> d
1152 {'x': [1]}
1153 >>> d2 = defaultdict(int)
1154 >>> d2['ghost']                                     # a mere read...
1155 0
1156 >>> 'ghost' in d2                                     # ...planted the key
1157 True
1158
1159 How it works: one dunder does everything. When d[k] misses, Python calls d.__missing__(k); this subclass
1160 responds by calling the stored factory, installing the result, and returning it. The factory runs only on
1161 a miss -- and it takes NO arguments, which is why the leaf recipes in Tree/ITree are zero-argument
1162 lambdas.
1163
1164 Why it earns its place: it deletes the check-then-create dance from every grouping and counting loop. The
1165 write path becomes a bare append or +=, and the structure assembles itself on touch -- autovivification,
1166 which section 12's trie tools then apply recursively.
1167
1168 The discipline it demands: a mere READ also plants the key, so split your paths -- write with d[k], but
1169 ask questions with 'in' or .get. The prelude's bag operations read Counters with .get for exactly this
1170 reason.
1171 >>> d = defaultdict(int)
1172 >>> d['hits'] += 1                                     # write path: indexing is the point
1173 >>> d.get('misses', 0), 'misses' in d                 # read path: no ghost key planted
1174 (0, False)
1175
1176 Remember it as: index to build, .get to ask.
1177
1178 Roll your own: subclass dict and implement one dunder -- __missing__, which Python calls when [k] fails.
1179 Call the factory, install the result, hand it back; iteration, repr and everything else is inherited.
1180 Note the factory takes no arguments, which is why leaf recipes are zero-argument lambdas.
1181
1182 Counter -- count occurrences: a defaultdict(int) fold over the input. Missing keys read as 0 (mind the
1183 autovivification gotcha above); rank the .items() with sortOn.
1184 >>> c = Counter("abracadabra")
1185 >>> c['a'], c['z']
1186 (5, 0)
1187 >>> sortOn(lambda kv: -kv[1], list(Counter("aab").items()))
1188 [('a', 2), ('b', 1)]
1189
1190 Roll your own: a defaultdict(int) plus one fold -- d[x] += 1 per element. The += on a missing key is
1191 autovivification doing all the work.
1192
1193 insertWith -- Data.Map's insertWith f k v m: insert (k, v), combining with f(new, old) on collision --
1194 Haskell's argument order, new value first. PERSISTENT, also like Haskell: it returns a fresh dict and
1195 leaves the argument untouched.
1196 >>> insertWith(add, 'x', 4, {'x': 1})
1197 {'x': 5}
1198 >>> d = {'y': 2}
1199 >>> insertWith(add, 'x', 4, d)
1200 {'y': 2, 'x': 4}
1201 >>> d                                                 # the original survives
1202 {'y': 2}
1203
1204 Roll your own: one dict display -- {**d, k: ...}: the splat copies d (that IS the persistence), and the
1205 new key's value is f(v, d[k]) on collision (new value first, Haskell's order) or plain v. Compare
1206 fromListWith, which applies the same rule but mutates, because there the dict is its own private
1207 accumulator.
1208
1209 fromListWith -- THE dict-building fold: pour (key, value) pairs into a dict, combining collisions with
1210 f(new, old) -- Haskell's order, like insertWith. Keys keep first-seen order.
1211 >>> fromListWith(add, [(c, 1) for c in "aab"])
1212 {'a': 2, 'b': 1}
1213 >>> fromListWith(lambda new, old: old + new, [('a', [1]), ('b', [2]), ('a', [3])])
1214 {'a': [1, 3], 'b': [2]}
1215
1216 The order gotcha, faithfully Haskell's: because f receives (new, old), grouping with a bare concat
1217 REVERSES each group -- the classic Data.Map surprise. Combine as old + new to keep arrival order, or use
1218 plain addition where order cannot matter (counts).
1219 >>> fromListWith(add, [(len(w), [w]) for w in ["hi", "ox", "sun"]])
1220 {2: ['ox', 'hi'], 3: ['sun']}
1221 >>> fromListWith(lambda new, old: old + new, [(len(w), [w]) for w in ["hi", "ox", "sun"]])
1222 {2: ['hi', 'ox'], 3: ['sun']}
1223
1224 Why it earns its place -- one function, three monoids: choosing f is choosing what the dict MEANS.
1225 Addition over (k, 1) pairs counts; old + new over (k, [v]) pairs groups; max merges bags. That is the
1226 whole family: insertWith is one collision, fromListWith a list of them, unionWith two dicts' worth (where
1227 f receives (a's value, b's value)).
1228
1229 The recognition cue: whenever a loop builds a dict with an if-else on key existence, you are hand-rolling
1230 this fold. Name the pairs, name the monoid, and the loop disappears.
1231
1232 Remember it as: grouping, counting, and merging are one fold -- whose combiner hears the NEW value first.

```

```

1233 Roll your own: an empty dict and one loop applying the collision rule per pair -- d[k] = f(v, d[k]) if
1234 the key exists, else v. It is insertWith inlined and IN-PLACE: mutation is fine here because the dict
1235 being built is the fold's own accumulator, invisible until returned.
1236
1237
1238 unionWith -- merge two dicts, combining SHARED keys with f; a's keys keep their order, b's new keys
1239 follow. Choosing f is choosing a monoid: max gives bag union, + gives a summing merge.
1240 >>> unionWith(max, {'a': 1, 'b': 5}, {'b': 2, 'c': 3})
1241 {'a': 1, 'b': 5, 'c': 3}
1242 >>> unionWith(add, {'a': 1, 'b': 2}, {'b': 3, 'c': 4})
1243 {'a': 1, 'b': 5, 'c': 4}
1244
1245 Roll your own: two splats -- (**a, **{...}): a copied whole, then overlaid with b's items after each has
1246 been through the collision rule, f(a[k], v) when k is shared, plain v when it is new. The asymmetry is
1247 deliberate: a's keys keep their positions, and only keys new to b append at the end.
1248
1249 group -- runs of CONSECUTIVE equal elements, as bare lists ([a]), no keys) -- Data.List's group.
1250 >>> group("aaabbc")
1251 [['a', 'a', 'a'], ['b', 'b'], ['c']]
1252 >>> group("abab")
1253 [['a'], ['b'], ['a'], ['b']]
1254
1255 groupBy -- runs by an equivalence predicate; each newcomer is compared with eq against the run's FIRST
1256 element (span-based, exactly Haskell's groupBy), not against its neighbour -- a subtle and deliberate
1257 fidelity.
1258 >>> groupBy(lambda a, b: a == b, [1, 1, 2])
1259 [[1, 1], [2]]
1260 >>> groupBy(lambda a, b: b - a <= 1, [1, 2, 3, 10, 11])
1261 [[1, 2], [3], [10, 11]]
1262
1263 The word that matters is CONSECUTIVE: a new run starts whenever eq against the run's first element fails,
1264 so 'abab' yields four runs. Not a weakness -- a different question: runs, streaks and compression care
1265 about position, and these are the only tools asking about it.
1266 >>> [(run[0], len(run)) for run in group("aaabbbcc")] # run lengths
1267 [('a', 3), ('b', 4), ('c', 2)]
1268
1269 For CATEGORIES -- position irrelevant -- either sort first, so equals become adjacent, or use
1270 fromListWith, which never cared about position at all. Choosing between them is stating what your key
1271 means.
1272 >>> [(run[0], len(run)) for run in group(sorted("abab"))]
1273 [('a', 2), ('b', 2)]
1274
1275 Remember it as: group answers "how long are the runs?"; fromListWith answers "what are the piles?".
1276
1277 Roll your own: one pass in which each element either joins the current run or starts a fresh one. The
1278 test is eq(out[-1][0], x) -- against the run's FIRST element, not its last -- which is exactly how
1279 Haskell's span-based groupBy behaves, and the detail that makes non-transitive predicates (like "within 1
1280 of") agree with it.
1281
1282 bag_union -- multisets (bags) are Counters; union takes the LARGER count per key. Key order follows set
1283 union, so compare with == rather than printing.
1284 >>> bag_union(Counter("aab"), Counter("abb")) == {'a': 2, 'b': 2}
1285 True
1286
1287 bag_inter -- intersection: the SMALLER count per shared key, zero counts dropped. Note the implementation
1288 reads with .get, never [] -- indexing a Counter would autovivify zeros into it.
1289 >>> bag_inter(Counter("aab"), Counter("ab")) == {'a': 1, 'b': 1}
1290 True
1291 >>> bag_inter(Counter("aa"), Counter("bb")) == {}
1292 True
1293
1294 bag_diff -- difference, clipped at zero: what remains of a after removing b.
1295 >>> bag_diff(Counter("aab"), Counter("ab")) == {'a': 1}
1296 True
1297
1298 bag_sub -- inclusion: is bag a contained in bag b, multiplicities included? The ransom-note test.
1299 >>> bag_sub(Counter("ab"), Counter("aabb"))
1300 True
1301 >>> bag_sub(Counter("aab"), Counter("ab"))
1302 False
1303
1304 Tree -- nested defaultdicts to a FIXED depth, with a leaf factory at the bottom. Tree(1, list) is the
1305 grouping dict; Tree(2, int) is a sparse 2-D counting table. Indexing autovivifies the whole path.
1306 >>> t = Tree(1, list)
1307 >>> t['evens'].append(2)
1308 >>> t
1309 {'evens': [2]}
1310 >>> m = Tree(2, int)
1311 >>> m['a']['b'] += 1
1312 >>> m
1313 {'a': {'b': 1}}
1314
1315 ITree -- the INFINITE tree: every branch is another ITree, so any depth autovivifies on touch. This is
1316 the trie: index letter by letter and the branch builds itself.
1317 >>> t = ITree()
1318 >>> t['x']['y']['z'] = 9
1319 >>> t
1320 {'x': {'y': {'z': 9}}}
```

```

1321
1322 How it works: the definition is a knot -- ITree = lambda: defaultdict(ITree). Every missing key
1323 manufactures another ITree, which will do the same, without end. Indexing therefore GROWS the tree:
1324 touching t['c']['a']['t'] conjures the whole branch. A trie in one line, built from nothing but
1325 __missing__.
1326 >>> t = ITree()
1327 >>> for w in ["cat", "car"]: setpath(t, list(w) + ['$'], w)
1328 >>> sorted([leaf for p, leaf in paths(t) if p[-1] == '$'])
1329 ['car', 'cat']
1330
1331 Why it earns its place: hierarchies -- tries, directory trees, nested groupings -- stop needing setup
1332 code. Writing is indexing (or setpath, which also plants a leaf); harvesting is paths. The one edged
1333 rule: reads autovivify too, so QUESTIONS go through getpath, never bare indexing -- getpath's entry
1334 demonstrates the ghost branch this avoids.
1335
1336 Versus Tree(depth, leaf): Tree is the finite cousin -- fixed depth, with a real leaf factory (list, int)
1337 at the bottom instead of more tree. Tree(1, list) is the grouping dict; ITree is for depth you cannot
1338 know in advance, like words.
1339
1340 Remember it as: a dict that dreams up more of itself on demand.
1341
1342 Roll your own: ITree = lambda: defaultdict(ITree) mentions itself -- legal, because the lambda body only
1343 runs on a MISS, by which time the name is bound. Tree bottoms the same recursion out at a fixed depth by
1344 handing the last level a real leaf factory instead of more tree.
1345
1346 paths -- fold a whole Tree/ITree into a flat list of (keypath, leaf) pairs, in insertion order. The read-
1347 everything companion to setpath's write-one-path.
1348 >>> t = ITree()
1349 >>> setpath(t, ['a', 'b'], 1); setpath(t, ['a', 'c'], 2)
1350 >>> paths(t)
1351 [(['a', 'b'], 1), (['a', 'c'], 2)]
1352
1353 How it works: structural recursion with two arms, exactly the case-arms-as-fold story. A non-dict (or
1354 empty node) is a LEAF: report the single row ([], leaf). A dict is a BRANCH: recurse into every subtree
1355 and prepend the branching key to each keypath that comes back. The whole tree flattens into (keypath,
1356 leaf) rows, insertion-ordered, losslessly.
1357
1358 Why it earns its place: building a trie is the easy half; the value is HARVESTING it. autocomplete
1359 filters rows under a prefix, deepest_directory takes maxOn of keypath length, leaves reads the second
1360 column. One cata, many harvests -- you never write tree-walking code again.
1361 >>> t = ITree()
1362 >>> setpath(t, ['a', 'b'], 1); setpath(t, ['c'], 2)
1363 >>> paths(t)
1364 [(['a', 'b'], 1), (['c'], 2)]
1365 >>> maxOn(lambda row: len(row[0]), paths(t))
1366 (['a', 'b'], 1)
1367
1368 Remember it as: paths turns a tree back into rows; setpath turns rows into a tree.
1369
1370 Roll your own: recursion on one question -- is this still a dict? A leaf reports a single row with an
1371 empty path; a branch recurses into every child and conses its key onto every path that comes back. Depth
1372 equals tree height, comfortably inside Python's limit for sane tries.
1373
1374 leaves -- just the leaf values of paths.
1375 >>> t = ITree()
1376 >>> setpath(t, ['a'], 1); setpath(t, ['b'], 2)
1377 >>> leaves(t)
1378 [1, 2]
1379
1380 setpath -- write a leaf value at the end of a key path, autovivifying the branch on the way. Caution:
1381 writing at an interior key CLOBBERS the subtree below it.
1382 >>> t = ITree()
1383 >>> setpath(t, ['a', 'b'], 7)
1384 >>> t
1385 {'a': {'b': 7}}
1386 >>> setpath(t, ['a'], 0) # clobbers {'b': 7}
1387 >>> t
1388 {'a': 0}
1389
1390 Roll your own: walk all but the last key by PLAIN INDEXING -- on an ITree that autovivifies the branch as
1391 you go -- then assign at the final key, the single write. getpath is the mirror image with the safety
1392 catch on: isinstance and 'in' checks instead of indexing, so reading plants nothing.
1393
1394 getpath -- read along a key path WITHOUT autovivifying, returning a default when the path is absent. This
1395 is why it exists: reading a trie with [] would plant ghost branches.
1396 >>> t = ITree()
1397 >>> setpath(t, ['a', 'b'], 7)
1398 >>> getpath(t, ['a', 'b']), getpath(t, ['a', 'z'], default=0)
1399 (7, 0)
1400 >>> 'z' in t['a'] # no ghost branch was planted
1401 False
1402
1403 deque -- a double-ended queue built from two stacks, all four end operations amortised O(1) (elements
1404 migrate between stacks only when one runs dry). This is the BFS frontier; a plain list's pop(0) is O(n).
1405 >>> d = deque([1, 2, 3])
1406 >>> d.appendleft(0); d.append(4)
1407 >>> d.popleft(), d.pop()
1408 (0, 4)

```

```

1409 >>> len(d), list(d)
1410 (3, [1, 2, 3])
1411
1412 How it works: two plain lists, nose to nose -- _out holds the front (reversed), _in holds the back.
1413 append pushes onto _in, appendleft onto _out, both O(1). The trick is popping an EMPTY side: the other
1414 list is reversed across in one O(n) migration, and popping is O(1) again.
1415
1416 Why that still counts as O(1): amortization. Each element migrates at most once between being pushed and
1417 popped, so n operations cost O(n) in total -- constant per operation on average, which is what the
1418 workload's big-O cares about. A bare Python list cannot say this: pop(0) shifts every element, every
1419 single time.
1420 >>> q = deque()
1421 >>> for x in [1, 2, 3]: q.append(x)
1422 >>> q.popleft(), q.popleft()          # first pop pays the migration; the next is free
1423 (1, 2)
1424
1425 Why it earns its place: BFS. The frontier must yield its OLDEST member (popleft) while newcomers join at
1426 the back (append) -- levelorder and grid_path are this class doing its one job.
1427
1428 Remember it as: two stacks facing each other make a queue; the reversal bill is paid once per element.
1429
1430 Roll your own: two plain lists nose to nose, each end appending and popping its own list at O(1). All the
1431 cleverness is in the pop guard: when your side is empty, reverse the OTHER list across in one go. Each
1432 element makes that crossing at most once between entering and leaving, so the occasional O(n) reversal
1433 amortises to O(1) per operation.
1434
1435 dsu -- disjoint-set union (union-find) over n elements, with path halving. find gives a set's
1436 representative; union merges and returns False when the two were ALREADY connected -- which is exactly a
1437 cycle detector (Kruskal's test).
1438 >>> find, union = dsu(4)
1439 >>> union(0, 1), union(1, 2)
1440 (True, True)
1441 >>> union(0, 2)          # already connected: a cycle
1442 False
1443 >>> find(0) == find(2), find(0) == find(3)
1444 (True, False)
1445
1446 How it works: parent[] links every element toward its set's ROOT, and find follows the chain up. The
1447 subtle line is parent[x] = parent[parent[x]] -- path HALVING: every find re-points each visited node at
1448 its grandparent, so chains shrink as a side effect of being walked. That one line makes operations
1449 effectively constant time (inverse Ackermann, for the curious).
1450
1451 Why it earns its place: dynamic connectivity. "Are these two in the same group yet?" under a stream of
1452 merges is awkward for every other structure and trivial here. And union returning False -- already
1453 connected -- IS cycle detection: Kruskal's spanning tree is built by skipping every False.
1454 >>> find, union = dsu(5)
1455 >>> [union(a, b) for a, b in [(0, 1), (1, 2), (3, 4), (0, 2)]]
1456 [True, True, True, False]
1457 >>> len({find(i) for i in range(5)})    # two components remain
1458 2
1459
1460 Remember it as: find walks to the root, halving as it goes; a False union is a cycle caught.
1461
1462 Roll your own: parent[] as the forest and two closures instead of a class. find's loop carries the entire
1463 trick in one line -- parent[x] = parent[parent[x]] re-points every visited node at its grandparent (path
1464 halving), flattening chains as a side effect of walking them. union is find twice plus one root re-point,
1465 answering False when the roots already agree.
1466
1467 heappush -- push onto a min-heap kept in a plain list (sift-up). The smallest element is always h[0]. For
1468 a MAX-heap, push negated values.
1469 >>> h = []
1470 >>> for x in [5, 1, 8]: heappush(h, x)
1471 >>> h[0]
1472 1
1473
1474 How it works: the flat list IS a binary tree -- h[i]'s children live at 2i+1 and 2i+2 -- kept under one
1475 invariant: every parent <= its children. heappush appends at the first free leaf and sifts UP, swapping
1476 with its parent while smaller; heappop moves the last leaf to the root and sifts DOWN toward the smaller
1477 child. Each walks one root-to-leaf path: O(log n).
1478
1479 Why the invariant is enough: nothing is fully sorted -- siblings sit in any order -- yet h[0] is ALWAYS
1480 the minimum, because the root beats its children, who beat theirs. You pay log n per operation for a
1481 permanently readable minimum; draining the heap is heapsort; pushing negatives turns min into max.
1482 >>> h = []
1483 >>> for x in [7, 2, 9, 4]: heappush(h, x)
1484 >>> h[0]
1485 2
1486 >>> [heappop(h) for _ in range(4)]
1487 [2, 4, 7, 9]
1488
1489 Why it earns its place: "repeatedly take the best next thing" -- Dijkstra's frontier, running_median's
1490 two halves, any streaming top-k. When next-best is needed many times while the data keeps changing, the
1491 heap beats re-sorting every time.
1492
1493 Remember it as: a flat list, a parent-beats-children promise, and log-n repairs.
1494
1495 Roll your own: append at the first free leaf, then sift UP -- while the parent at (i - 1) // 2 is larger,
1496 swap and climb. The index arithmetic IS the tree; watch the array after four pushes:

```

```

1497 >>> h = []
1498 >>> for x in [7, 2, 9, 4]: heappush(h, x)
1499 >>> h
1500 [2, 4, 9, 7]
1501
1502 heappop -- remove and return the minimum (sift-down). Popping repeatedly drains in sorted order -- that
1503 is heapsort.
1504 >>> h = []
1505 >>> for x in [5, 1, 8]: heappush(h, x)
1506 >>> [heappop(h) for _ in range(3)]
1507 [1, 5, 8]
1508 >>> g = []
1509 >>> for x in [3, 9, 5]: heappush(g, -x)
1510 >>> -heappop(g)
1511 9
1512
1513 Roll your own: swap the root with the last leaf, pop it off the end, then sift DOWN -- repeatedly swap
1514 with the SMALLER of the two children (choosing the smaller is the classic bug site) until neither child
1515 is smaller. One root-to-leaf path each way: O(log n).
1516
1517 ===== 13. nodes =====
1518
1519 TreeNode -- the LeetCode binary tree: val, left, right, with None for absent children.
1520 >>> t = TreeNode(2, TreeNode(1), TreeNode(3))
1521 >>> t.left.val, t.val, t.right.val
1522 (1, 2, 3)
1523
1524 tfold -- the tree catamorphism: recursively combines each node's value with the already-folded left and
1525 right results, z standing in for empty subtrees. EVERY structural tree computation is one choice of f and
1526 z -- the five traversal/measure functions below are all tfold one-liners.
1527 >>> t = TreeNode(2, TreeNode(1), TreeNode(3))
1528 >>> tfold(lambda v, l, r: v + l + r, 0, t)
1529 6
1530 >>> tfold(lambda v, l, r: max(v, l, r), 0, t)
1531 3
1532
1533 How it works: the tree catamorphism -- structural recursion with a contract. Two arms, exactly like a
1534 case expression: the empty tree answers z; a node answers f(value, left_result, right_result), both
1535 results computed by the same recursion. Hand it (f, z) and any question about a tree becomes one line --
1536 the five traversals and both measures in section 13 are all tfold with different arms.
1537
1538 Why it earns its place: it separates WALKING from DECIDING. The recursion is written once, correct
1539 forever; you only ever author the arms. Even tree TRANSFORMATION fits -- mirror a tree by swapping the
1540 child results while rebuilding:
1541 >>> t = TreeNode(2, TreeNode(1), TreeNode(3))
1542 >>> mirror = tfold(lambda v, l, r: TreeNode(v, r, l), None, t)
1543 >>> inorder(mirror)
1544 [3, 2, 1]
1545
1546 The one caveat: recursion depth equals tree height -- fine for balanced and interview-sized trees; say a
1547 sentence out loud about adversarially skewed ones.
1548
1549 Remember it as: z is the empty-tree arm, f is the node arm -- the fold walks, you decide.
1550
1551 Roll your own: one line of structural recursion -- z when the tree is None, else f over the node's value
1552 and the two recursive results.
1553
1554 Roll your own: one line of structural recursion -- z when the tree is None, else f over the node's value
1555 and the two recursive results. Every traversal and measure in this section is that line with a different
1556 f plugged in.
1557
1558 inorder -- left, node, right; on a binary SEARCH tree this is sorted order.
1559 >>> inorder(TreeNode(2, TreeNode(1), TreeNode(3)))
1560 [1, 2, 3]
1561
1562 preorder -- node first, then children: the copy/serialise order.
1563 >>> preorder(TreeNode(2, TreeNode(1), TreeNode(3)))
1564 [2, 1, 3]
1565
1566 postorder -- children first, then node: the delete/evaluate order.
1567 >>> postorder(TreeNode(2, TreeNode(1), TreeNode(3)))
1568 [1, 3, 2]
1569
1570 tdepth -- the height of the tree.
1571 >>> tdepth(TreeNode(2, TreeNode(1), TreeNode(0)), TreeNode(3)))
1572 3
1573
1574 tsize -- the number of nodes.
1575 >>> tsize(TreeNode(2, TreeNode(1), TreeNode(3)))
1576 3
1577
1578 levelorder -- breadth-first values, level by level: the deque earning its keep. Compare preorder on the
1579 same tree to see DFS vs BFS.
1580 >>> t = TreeNode(1, TreeNode(2, TreeNode(4)), TreeNode(3))
1581 >>> levelorder(t)
1582 [1, 2, 3, 4]
1583 >>> preorder(t)
1584 [1, 2, 4, 3]

```

```

1585
1586 Roll your own: seed a deque with the root, then loop -- popleft, record the value, append the children
1587 that exist. FIFO order IS breadth-first; swap the deque for a list used as a stack and the identical loop
1588 turns into depth-first.
1589
1590 ListNode -- the LeetCode singly-linked list: val and next.
1591 >>> n = ListNode(1, ListNode(2))
1592 >>> n.val, n.next.val
1593 (1, 2)
1594
1595 from_list -- build a linked list from a Python list: literally foldr(ListNode, xs, None). foldr's f takes
1596 (element, accumulator), and ListNode(val, next) has exactly that shape, so the last element is wrapped
1597 first and each earlier one points at the list built so far.
1598 >>> to_list(from_list([1, 2, 3]))
1599 [1, 2, 3]
1600
1601 to_list -- walk a linked list back into a Python list; the round-trip partner of from_list, and the way
1602 to make node results printable. An unfoldr: the seed is the node, each step yields (node.val, node.next),
1603 and None ends it.
1604 >>> to_list(from_list("ab"))
1605 ['a', 'b']
1606 >>> to_list(None)
1607 []
1608
1609 reverse_list -- reverse in place with the three-pointer walk; prev is a fold accumulator wearing
1610 pointers.
1611 >>> to_list(reverse_list(from_list([1, 2, 3])))
1612 [3, 2, 1]
1613
1614 Roll your own: the three-pointer walk compressed into one simultaneous assignment -- node.next, prev,
1615 node = prev, node.next. The right-hand side is evaluated in full before any assignment lands, which
1616 is what saves the pointer you are about to overwrite.
1617
1618 middle -- the middle node by fast/slow pointers (fast moves twice per step); on even lengths, the SECOND
1619 middle.
1620 >>> middle(from_list([1, 2, 3])).val
1621 2
1622 >>> middle(from_list([1, 2, 3, 4])).val
1623 3
1624
1625 Roll your own: fast advances two hops for slow's one, so when fast falls off the end, slow stands at the
1626 middle. The loop condition (fast and fast.next) is precisely what selects the SECOND middle on even
1627 lengths.
1628
1629 has_cycle -- Floyd's tortoise and hare: the fast pointer laps the slow one if and only if a cycle exists;
1630 O(1) space.
1631 >>> has_cycle(from_list([1, 2, 3]))
1632 False
1633 >>> n = ListNode(1); n.next = n # a self-loop
1634 >>> has_cycle(n)
1635 True
1636
1637 Roll your own: the same fast/slow walk, but watching for the pointers to MEET -- on a cycle the fast
1638 runner laps the slow one; on a straight list it simply falls off. O(1) space, no visited set.
1639
1640 merge_lists -- merge two SORTED linked lists, using the dummy-head idiom to avoid special-casing the
1641 first node.
1642 >>> to_list(merge_lists(from_list([1, 3]), from_list([2, 4])))
1643 [1, 2, 3, 4]
1644
1645 Roll your own: the dummy-head idiom -- a throwaway node to hang the result from, so appending the first
1646 real node needs no special case; walk both lists copying the smaller head, glue the survivor, return
1647 dummy.next.
1648
1649 ===== 14. grids & windows =====
1650
1651 neighbors4 -- the in-bounds orthogonal neighbours of cell (r, c) in an R-by-C grid. Bounds checking lives
1652 HERE, so BFS bodies stay clean.
1653 >>> sorted(neighbors4(0, 0, 2, 2)) # corner: two neighbours
1654 [(0, 1), (1, 0)]
1655 >>> sorted(neighbors4(1, 1, 3, 3)) # centre: four
1656 [(0, 1), (1, 0), (1, 2), (2, 1)]
1657
1658 Roll your own: yield each in-bounds delta of (r, c) -- the bounds check lives HERE so every caller's BFS
1659 body stays clean. neighbors8 is the same generator over a 3x3 delta grid, with (dr or dc) excluding the
1660 centre cell.
1661
1662 neighbors8 -- the same plus diagonals.
1663 >>> len(list(neighbors8(1, 1, 3, 3)))
1664 8
1665 >>> sorted(neighbors8(0, 0, 2, 2))
1666 [(0, 1), (1, 0), (1, 1)]
1667
1668 longest_window -- the sliding-window skeleton: it grows the right edge one element at a time (calling
1669 add), shrinks the left edge while your valid() says the window is illegal (calling rem), and tracks the
1670 best length. YOU supply the state as closures; the skeleton does the two-pointer bookkeeping. Longest-
1671 substring-without-repeats and longest-run-under-a- budget are both three closures away.
1672 >>> seen = []

```

```

1673 >>> longest_window("abcabb", lambda: len(seen) == len(set(seen)),
1674 ...                 seen.append, lambda c: seen.pop(0))
1675 3
1676 >>> w = []
1677 >>> longest_window([2, 1, 3, 4], lambda: sum(w) <= 6,
1678 ...                 w.append, lambda x: w.pop(0))
1679 3
1680
1681 How it works: inversion of control. The skeleton owns the two pointers -- it grows hi one element per
1682 step (calling add), then shrinks lo while your valid() reports the window broken (calling rem per evicted
1683 element) -- and it tracks the best length ever seen legal. You own only the STATE, held in closures over
1684 variables sitting next to the call.
1685
1686 Designing the closures is answering one question: "what makes a window ILLEGAL, and what must I track to
1687 know?" Distinct characters -> a duplicate count. A budget -> a running sum. At most k of something -> a
1688 counter. add and rem are that state's increment and decrement; valid is the legality test.
1689 >>> w = []
1690 >>> longest_window("aabcb", lambda: len(w) == len(set(w)), w.append, lambda c: w.pop(0))
1691 3
1692
1693 Why closures and not a class: the state lives exactly as long as the call, sits in scope where you can
1694 read it, and needs no ceremony. Functions carrying state -- the same muscle as curry's captured argument,
1695 here working a two-pointer job.
1696
1697 Remember it as: the skeleton slides the window; your three closures say what legal means.
1698
1699 Roll your own: hi sweeps forward exactly once, add() greeting each newcomer; an inner while shrinks lo,
1700 rem() by rem(), until valid() approves again. Every element is added once and removed at most once -- the
1701 entire two-pointer O(n) argument, visible in two loops.
1702
1703 ===== 15. control =====
1704
1705 memo -- an unbounded memoizer as a plain decorator (no parentheses): results are cached by the argument
1706 tuple, so each distinct call computes once. Keyword arguments fold into the key; the cache is exposed as
1707 .cache.
1708 >>> @memo
1709 ... def fib(n): return n if n < 2 else fib(n - 1) + fib(n - 2)
1710 >>> fib(30)
1711 832040
1712 >>> fib.cache[(10,)]
1713 55
1714 >>> len(fib.cache)                                # one entry per distinct argument
1715 31
1716
1717 How it works: the decorator wraps f with a dict keyed on the arguments. First call computes and stores;
1718 every repeat is a lookup. That is the entire mechanism -- and why arguments must be HASHABLE: lists
1719 become tuples before they can be keys.
1720
1721 Why it earns its place: memoized recursion IS dynamic programming. Write the honest recurrence -- the
1722 answer for state (i, j) in terms of smaller states -- decorate it, and the cache is your DP table, filled
1723 in dependency order with no index bookkeeping. The exposed .cache lets you SEE the table: its size is the
1724 state count, exactly the number complexity analysis wants said out loud.
1725 >>> @memo
1726 ... def ways(r, c):                                # lattice paths to (r, c), moving right/down
1727 ...     return 1 if r == 0 or c == 0 else ways(r - 1, c) + ways(r, c - 1)
1728 >>> ways(3, 3)
1729 20
1730 >>> len(ways.cache)                                # the 4x4 grid minus (0,0), never needed
1731 15
1732
1733 Versus functools.lru_cache: the standard library's version adds eviction (maxsize); memo is deliberately
1734 unbounded, because DP wants every state kept -- and it is nine lines you can retype in any interview that
1735 bans imports.
1736
1737 Remember it as: recursion states the truth; the cache makes it affordable; .cache is the table.
1738
1739 Roll your own: a dict captured in a closure, keyed by the argument tuple (keyword arguments folded in as
1740 a frozenset when present -- tuples and frozensets because keys must be hashable). Compute on miss, look
1741 up on hit, and hang the dict on the wrapper as .cache so the DP table stays inspectable.
1742
1743 until -- iterate f from x until the predicate holds, returning the first satisfying value: a fixed-point
1744 loop with no recursion-depth limit. The state can be a tuple carrying whatever the loop must remember.
1745 >>> until(lambda x: x > 100, lambda x: x * 2, 1)
1746 128
1747 >>> collatz = lambda s: (s[0] // 2 if even(s[0]) else 3 * s[0] + 1, s[1] + 1)
1748 >>> until(lambda s: s[0] == 1, collatz, (6, 0))      # 8 steps to reach 1
1749 (1, 8)
1750
1751
1752 How it works: keep applying f until p approves -- iteration to a goal or a FIXED POINT, as a loop rather
1753 than recursion, so depth is unbounded. The state can be anything; a tuple lets one loop carry several
1754 facts at once, like the collatz example's (value, steps).
1755
1756 Why it earns its place: some processes have no list to fold over -- they run "until stable". Newton's
1757 isqrt converges; bellman_ford relaxes every edge until the distance map stops changing (the textbook
1758 fixed point; g7's hint calls it by name). until states such loops declaratively: here is the step, here
1759 is done.
1760 >>> until(lambda n: n * n >= 30, succ, 0)            # smallest n with n squared >= 30

```

```
1761 6
1762 >>> until(lambda xs: xs == sorted(xs), sorted, [3, 1, 2])
1763 [1, 2, 3]
1764
1765 Remember it as: while-not-done, promoted to an expression that returns its answer.
1766
1767 Roll your own: while not p(x): x = f(x); return x. Iteration promoted to a value-returning expression --
1768 and a loop rather than recursion on purpose, because a fixed point may take unboundedly many steps."""
1769
1770 from prelude import *                                # the one permitted import
1771
1772 add = lambda a, b: a + b                             # used throughout the examples
1773
1774
1775 if __name__ == u'__main__':
1776     import doctest
1777     r = doctest.testmod()
1778     print(f"prelude-doctests: {r.attempted - r.failed}/{r.attempted} examples passing")
end
```